

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++

Juan Carlos Morales
Doble titulación: Sistemas & Robótica

Bogotá, Colombia · 2026

Índice

I	Fundamentos Básicos	3
1	Entrada / Salida Rápida (Fast I/O)	3
2	Formateo de Salida	3
3	Condicionales y Ciclos	4
4	Conversiones, Parseos y Casteos	4
II	Estructuras de Datos	5
5	Basic Structures	5
5.1	Matrices (Arreglos 2D)	5
5.2	Vectores / Listas dinámicas	5
5.3	Listas enlazadas (LinkedList)	5
5.4	Pilas (Stack)	5
5.5	Colas (Queue)	5
5.6	Colas de prioridad (Priority Queue / Heap)	6
5.7	Vector circular (Circular Buffer)	6
6	Estructuras Hash/Tree	7
6.1	Conjuntos hash (HashSet)	7
6.2	Mapas / Diccionarios hash	7
6.3	Conjuntos ordenados (TreeSet)	8
6.4	Mapas ordenados (TreeMap)	8
7	Trees	9
8	Range Queries	9
8.1	Casos de uso	9
8.2	Prefix Sums	10
8.3	Suma de Euler Totient (ϕ) en un rango	10
8.4	Árbol de Fenwick (BIT)	11
8.5	Sqrt Decomposition	13
8.6	Sparse Table (RMQ)	13
8.7	Segment Tree	14
8.8	RMQ (Range Minimum Query)	14
8.9	Mo's Algorithm	14
8.10	Wavelet Tree	14
8.11	Heavy-Light Decomposition	14
III	Grafos y Árboles	14
9	Grafos (Graphs)	14
10	Árboles (Trees)	14
11	Disjoint Set Union (DSU)	14
IV	Ordenamiento y Búsqueda	14
12	Ordenamiento y Búsqueda (Sorting & Searching)	14
12.1	Número faltante (Missing Number)	14
12.2	Máximo / Mínimo (Array Max/Min)	14
12.3	Par de suma absoluta mínima	14
12.4	Par con diferencia dada (Difference Pair)	14
12.5	Búsqueda ternaria (Ternary Search)	15

12.6	Búsqueda de Fibonacci (Fibonacci Search)	15
12.7	Búsqueda exponencial (Exponential Search)	15
12.8	Búsqueda por saltos (Jump Search)	15
12.9	Heap Sort	15
12.10	QuickSelect	15
13	Búsqueda Binaria (Binary Search)	15
13.1	En arreglo ordenado	16
13.2	Menor x que cumple	16
13.3	Mayor x que cumple	16
13.4	Sobre reales	16
13.5	Con strings	16
V	Matemáticas	16
14	Matemáticas (Math)	16
15	Teoría de Números (Number Theory)	16
16	Combinatoria (Combinatorics)	16
17	Máscaras de Bits (Bitmasking)	16
18	Geometría (Geometry)	16
VI	Técnicas Algorítmicas	16
19	Programación Dinámica (Dynamic Programming)	16
20	Algoritmos Voraces (Greedy)	16
21	Ad-hoc y Simulación	16
VII	Cadenas	16
22	Cadenas (Strings)	16

Parte I Fundamentos Básicos

1 Entrada / Salida Rápida (Fast I/O)

Usar lectura rápida para evitar TLE, hay 3 niveles de lectura rápida, estandarizar Tier 2 y usar Tier 3 (lector por bytes) solo cuando haya del orden de 10^6 – 10^7 lecturas.

Table 1: Casos donde se recomienda Fast I/O

Tipo de problema	Ejemplo
Arrays gigantes	Leer/ordenar/sumar 10^6 – 10^7 enteros
Muchos test cases (T grande)	$T \leq 10^5$, cada uno trivial pero se acumula
Grafos grandes	$M \sim 10^6$ – $2 \cdot 10^6$ aristas para BFS/DFS/Dijkstra/MST
Geometría con muchas consultas	L, S grandes + muchos test cases seguidos hasta EOF
Muchos tokens de texto	Strings/lineas que suman millones de caracteres
Judges clásicos "trampa de I/O"	SPOJ INTEST, ARRAY-SUB, etc.

Regla practica: si N/M es del orden de 10^4 – 10^5 , cin con `sync_with_stdio(false)`; `cin.tie(nullptr)`; alcanza de sobra. El FastReader es la "artillería pesada" para cuando eso deja de ser suficiente.

Tier 1 — Básico: cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

Tier 2 — Rápido: desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

Tier 3 — Más rápido: lector por bytes con fread. Solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leidos = 0;
5
6     int leer() {
7         if (puntero == leidos) {
8             leidos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leidos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13    int nextInt() {
14        int resultado = 0, b = leer();
15        while (b <= ' ') b = leer(); // saltar blancos
16        bool negativo = (b == '-');
17        if (negativo) b = leer();
18        while (b >= '0' && b <= '9') {
19            resultado = resultado * 10 + (b - '0'); b = leer();
20        }
21        return negativo ? -resultado : resultado;
22    }
23    long long nextLong() {
24        long long resultado = 0; int b = leer();
25        while (b <= ' ') b = leer();
```

```
26        bool negativo = (b == '-');
27        if (negativo) b = leer();
28        while (b >= '0' && b <= '9') {
29            resultado = resultado * 10 + (b - '0'); b = leer();
30        }
31        return negativo ? -resultado : resultado;
32    }
33    double nextDouble() {
34        double resultado = 0, divisor = 1; int b = leer();
35        while (b <= ' ') b = leer();
36        bool negativo = (b == '-');
37        if (negativo) b = leer();
38        while (b >= '0' && b <= '9') {
39            resultado = resultado * 10 + (b - '0'); b = leer();
40        }
41        if (b == '.') {
42            b = leer();
43            while (b >= '0' && b <= '9') {
44                resultado += (b - '0') / (divisor *= 10); b = leer();
45            }
46        }
47        return negativo ? -resultado : resultado;
48    }
49    string next() { // un token
50        string s; int b = leer();
51        while (b <= ' ') b = leer();
52        while (b > ' ') { s += (char) b; b = leer(); }
53        return s;
54    }
55    string nextLine() { // línea completa
56        string s; int b = leer();
57        while (b != '\n' && b != -1) {
58            if (b != '\r') s += (char) b; b = leer();
59        }
60        return s;
61    }
62 };
```

2 Formateo de Salida

La función `printf` utiliza una cadena de formato para indicar cómo debe interpretarse y mostrarse cada argumento. Cada especificador comienza con `%` y corresponde al argumento que aparece después de la cadena.

```
1 // %d -> entero (int)
2 printf("%d\n", 42);
3 // Entrada: 42
4 // Salida: 42
5
6 // %lld -> entero largo (long long)
7 long long x = 1234567890123LL;
8 printf("%lld\n", x);
9 // Entrada: 1234567890123
10 // Salida: 1234567890123
11
12 // %5d -> ancho mínimo de 5 caracteres, alineado a la derecha
13 printf("%5d\n", 42);
14 // Entrada: 42
15 // Salida: "  42"
16
17 // %-5d -> ancho mínimo de 5 caracteres, alineado a la izquierda
18 printf("%-5d\n", 42);
19 // Entrada: 42
20 // Salida: "42  "
21
22 // %05d -> ancho 5, completando con ceros
23 printf("%05d\n", 42);
24 // Entrada: 42
25 // Salida: "00042"
26
27 // %+d -> muestra siempre el signo
28 printf("%+d\n", 42);
29 // Entrada: 42
30 // Salida: "+42"
31
32 // %x -> hexadecimal usando letras minúsculas
33 printf("%x\n", 255);
34 // Entrada: 255
35 // Salida: ff
36
37 // %X -> hexadecimal usando letras mayúsculas
38 printf("%X\n", 255);
39 // Entrada: 255
40 // Salida: FF
41
42 // %.2f -> número real con exactamente 2 decimales
43 printf("%.2f\n", 3.14159);
```

```

44 // Entrada: 3.14159
45 // Salida: 3.14
46
47 // %.0f -> numero real sin decimales (redondea)
48 printf("%.0f\n", 3.7);
49 // Entrada: 3.7
50 // Salida: 4
51
52 // %e -> notacion cientifica
53 printf("%e\n", 31415.9);
54 // Entrada: 31415.9
55 // Salida: 3.141590e+04
56
57 // %c -> un solo caracter
58 printf("%c\n", 'A');
59 // Entrada: 'A'
60 // Salida: A
61
62 // %s -> cadena de caracteres
63 printf("%s\n", "hola");
64 // Entrada: "hola"
65 // Salida: hola
66
67 // Se pueden combinar varios especificadores
68 printf("%d %lld %.2f\n", 42, 10000000000LL, 3.14159);
69 // Entrada: 42, 10000000000, 3.14159
70 // Salida: 42 10000000000 3.14
71
72 // %% -> imprime un '%' literal
73 printf("100%%\n");
74 // Entrada: no recibe ningun argumento para %
75 // Salida: 100%
76
77 // ! NO uses %n en C/C++ (puede ser peligroso)
78 // Para scanf, un double se lee con %lf

```

Estructura de un especificador

Un formato puede combinar varias partes:

```

1 // % [flags] [width] [.precision] [specifier]
2 // % 0 5 .2 f
3
4 printf("%05d\n", 42);
5 // % -> comienza el formato
6 // 0 -> rellena con ceros
7 // 5 -> ancho minimo de 5 caracteres
8 // d -> interpreta el argumento como int
9 //
10 // Salida: 00042

```

Los modificadores más utilizados en programación competitiva son:

Formato	Tipo	Ejemplo de salida
%d	int	42
%lld	long long	1234567890123
%c	char	A
%s	string/char*	hola
%f	double	3.140000
%.2f	double	3.14
%x	int	ff
%X	int	FF

Regla práctica: en programación competitiva, los formatos más importantes son %d para int, %lld para long long, %c para caracteres, %s para cadenas y %.2f (o una precisión diferente) para números reales.

3 Condicionales y Ciclos

```

1 // if / else: ejecuta un bloque segun una condicion
2 if (x > 0) cout << "pos"; // x > 0 -> "pos"
3 else if (x < 0) cout << "neg"; // x < 0 -> "neg"
4 else cout << "cero"; // x == 0 -> "cero"
5
6 // Operador ternario: condicion ? valor_si_true : valor_si_false
7 string s = (x >= 0) ? "no neg" : "neg";
8 // Si x >= 0, s = "no neg"; si no, s = "neg"
9

```

```

10 // switch: compara una expresion con diferentes valores
11 // Solo funciona directamente con tipos integrales como int o
12 // char,
13 // NO con string.
14 switch (x) {
15     case 1: /*...*/ break; // x == 1
16     case 2: case 3: /*...*/ break; // x == 2 o x == 3
17     default: /*...*/; // ningun case coincide
18 }
19 // ! Sin break, se continua ejecutando el siguiente case (fall-through)
20
21 // C++17: se puede declarar una variable dentro del if
22 if (int r = x % 2; r == 0) {}
23 // r solo existe dentro del if y sus bloques asociados
24
25 // for clasico: usa un indice y se repite mientras i < n
26 for (int i = 0; i < n; i++) {}
27
28 // range-based for: recorre directamente los elementos
29 // No proporciona el indice.
30 for (int v : datos) {}
31
32 // & obtiene una referencia al elemento original.
33 // Por tanto, las modificaciones afectan al vector.
34 for (int &v : datos) v *= 2;
35
36 // while: ejecuta mientras la condicion sea verdadera
37 while (c < n) c++;
38
39 // do-while: primero ejecuta el bloque y DESPUES verifica la
40 // condicion.
41 // Por eso se ejecuta AL MENOS una vez.
42 do {
43     c--;
44 } while (c > 0);
45
46 // break -> termina inmediatamente el ciclo o switch
47 // continue -> salta a la siguiente iteracion del ciclo
48
49 // ! C++ NO tiene break con etiqueta.
50 // Para salir de ciclos anidados se puede usar una bandera,
51 // encapsular la logica en una funcion y usar return, etc.

```

4 Conversiones, Parseos y Casteos

```

1 // string -> numero (parseo)
2 // Convierte una cadena de texto en el tipo numerico indicado.
3 int e = stoi("42");
4 // Entrada: "42" -> Salida: 42
5
6 long long l = stoll("10000000000");
7 // Entrada: "10000000000" -> Salida: 10000000000
8
9 double d = stod("3.14");
10 // Entrada: "3.14" -> Salida: 3.14
11
12 // Tambien se puede indicar la base numerica.
13 int bin = stoi("1010", nullptr, 2);
14 // "1010" en base 2 -> 10
15
16 int hex = stoi("ff", nullptr, 16);
17 // "ff" en base 16 -> 255
18
19
20 // numero -> string
21 // Convierte cualquier numero compatible en texto.
22 string s = to_string(42);
23 // Entrada: 42 -> Salida: "42"
24
25
26 // Casteo: fuerza la conversion de un tipo a otro.
27 // Al pasar de decimal a entero, se TRUNCA hacia 0.
28 int t = (int) 3.99;
29 // 3.99 -> 3
30
31 int r = static_cast<int>(3.99);
32 // 3.99 -> 3
33 // static_cast<int> es la forma recomendada en C++
34
35
36 // ! int / int = division ENTERA
37 double mal = 7 / 2;
38 // 7 y 2 son int -> 3
39 // Luego 3 se convierte a double -> 3.0
40
41 double bien = 7 / 2.0;
42 // 2.0 es double -> division decimal
43 // 7 / 2.0 -> 3.5
44

```

```

45 // caracteres
46 // Los char tienen un valor numerico asociado (ASCII).
47 int cod = (int) 'A';
48 // 'A' -> 65
49
50
51 char ch = (char) 66;
52 // 66 -> 'B'
53
54 // Truco muy usado para convertir un digito char a int.
55 int dig = '7' - '0';
56 // '7' tiene valor 55 y '0' tiene valor 48
57 // 55 - 48 -> 7

```

Parte II Estructuras de Datos

5 Basic Structures

```

1 int a[5] = {1,2,3,4,5}; // crear con valores: O(n)
2
3 int b[5];                // crear de tamaño 5
4                          // posiciones: b[0] ... b[4]
5                          // ! Valores no inicializados
6
7 b[0] = 10;               // INSERT: O(1)
8 b[1] = 20;               // INSERT: O(1)
9
10 int x = b[0];            // READ: O(1)
11
12 b[0] = 30;               // UPDATE: O(1)
13
14 for (int i = 0; i < 5; i++)
15     cout << b[i] << " "; // READ: recorrer -> O(n)
16
17 // DELETE: no existe metodo para eliminar
18 // Se puede sobrescribir o manejar logicamente
19 b[0] = 0;                // DELETE logico: O(1)
20
21 // ! El tamaño de un array estatico no cambia.
22 // Para mas posiciones se necesita otro arreglo.

```

5.1 Matrices (Arreglos 2D)

Arreglo bidimensional (filas $\times$ columnas). **Complejidad:** acceso $[i][j]$ $O(1)$; crear $n \times m$ en $O(nm)$.

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // CREATE: O(n*m)
2 // matriz 3x4, todas las posiciones inicializadas en 0
3
4 m[1][2] = 7;             // INSERT: O(1)
5
6 int x = m[1][2];         // READ: O(1)
7
8 m[1][2] = 10;            // UPDATE: O(1)
9
10 for (int i = 0; i < 3; i++)
11     for (int j = 0; j < 4; j++)
12         cout << m[i][j] << " ";
13 // READ: recorrer toda la matriz -> O(n*m)
14
15 // DELETE: no existe metodo para eliminar una posicion.
16 // Se puede sobrescribir el valor o manejarlo logicamente.
17 m[1][2] = 0;             // DELETE logico: O(1)
18
19 // ! El tamaño de la matriz puede cambiar con vector,
20 // pero un arreglo 2D estatico no puede cambiar de tamaño.

```

5.2 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice $O(1)$; agregar al final $O(1)$ amortizado; insertar/borrar en medio $O(n)$; buscar $O(n)$.

```

1 vector<int> v;            // CREATE: lista vacia
2
3 v.push_back(3);          // INSERT: final -> O(1) amortizado
4 v.push_back(7);          // INSERT: final -> O(1) amortizado
5
6 int x = v[0];            // READ: acceso -> O(1)

```

```

7 int n = v.size();        // READ: tamaño -> O(1)
8
9 v[0] = 10;               // UPDATE: modificar -> O(1)
10
11 v.insert(v.begin() + 1, 5); // INSERT: medio -> O(n)
12
13 v.erase(v.begin() + 1);  // DELETE: medio -> O(n)
14
15 // Buscar un elemento recorriendo el vector
16 for (int x : v)
17     if (x == 10) { /* ... */ }
18 // SEARCH: recorrer -> O(n)
19
20 // DELETE: eliminar el ultimo elemento
21 v.pop_back();            // DELETE: final -> O(1)
22
23 // ! vector mantiene acceso por índice O(1).
24 // ! Al crecer puede realocar su memoria internamente.

```

5.3 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos** $O(1)$; acceso por índice $O(n)$; buscar $O(n)$. Útil como cola o deque.

```

1 list<int> l;              // CREATE: lista vacia
2
3 l.push_front(1);          // INSERT: inicio -> O(1)
4 l.push_back(9);           // INSERT: final -> O(1)
5
6 int x = l.front();         // READ: primer elemento -> O(1)
7 int y = l.back();          // READ: ultimo elemento -> O(1)
8
9 l.front() = 5;             // UPDATE: primer elemento -> O(1)
10 l.back() = 10;            // UPDATE: ultimo elemento -> O(1)
11
12 l.pop_front();            // DELETE: inicio -> O(1)
13 l.pop_back();             // DELETE: final -> O(1)
14
15 // ! No existe acceso directo por índice.
16 // Avanzar hasta una posicion requiere recorrer nodos.
17 auto it = l.begin();
18 advance(it, 2);           // acceso al tercer elemento -> O(n)
19
20 // Buscar un elemento requiere recorrer la lista.
21 for (int x : l)
22     if (x == 10) { /* ... */ }
23 // SEARCH: recorrer -> O(n)
24
25 // ! list no permite l[i].
26 // Para acceso por índice, vector suele ser mejor.

```

5.4 Pilas (Stack)

Estructura **LIFO** (último en entrar, primero en salir). **Complejidad:** insertar, eliminar y consultar el tope $O(1)$. Para DFS iterativo, paréntesis balanceados y deshacer.

```

1 stack<int> pila;          // CREATE: pila vacia
2
3 pila.push(3);             // INSERT: O(1)
4 pila.push(7);             // INSERT: O(1)
5
6 int tope = pila.top();     // READ: consultar tope -> O(1)
7
8 pila.top() = 10;          // UPDATE: modificar tope -> O(1)
9
10 pila.pop();               // DELETE: eliminar tope -> O(1)
11
12 // ! Solo se puede acceder directamente al elemento superior.
13 // ! No existe acceso por índice.

```

5.5 Colas (Queue)

Estructura **FIFO** (primero en entrar, primero en salir). **Complejidad:** encolar y desencolar $O(1)$. Base del BFS.

```

1 queue<int> cola;          // CREATE: cola vacia
2
3 cola.push(3);             // INSERT: final -> O(1)

```

```

4 cola.push(7);           // INSERT: final -> O(1)
5
6 int frente = cola.front(); // READ: primero -> O(1)
7 int atrás = cola.back();  // READ: ultimo -> O(1)
8
9 cola.front() = 10;       // UPDATE: modificar primero -> O(1)
10
11 cola.pop();             // DELETE: primero -> O(1)
12
13 // ! push agrega al final y pop elimina del frente.
14 // ! No existe acceso directo por índice.

```

5.6 Colas de prioridad (Priority Queue / Heap)

Devuelve siempre el elemento de mayor (o menor) prioridad. **Complejidad:** insertar y extraer $O(\log n)$; consultar el tope $O(1)$. Para Dijkstra, Prim, k-ésimo mayor.

```

1 priority_queue<int> pq; // CREATE: MAX-heap por defecto
2
3 pq.push(3);           // INSERT: O(log n)
4 pq.push(7);           // INSERT: O(log n)
5
6 int mx = pq.top();     // READ: mayor -> O(1)
7
8 pq.pop();             // DELETE: mayor -> O(log n)
9
10 // min-heap: el menor queda en el tope
11 priority_queue<int, vector<int>, greater<int>> pqMin;
12
13 pqMin.push(3);        // INSERT: O(log n)
14 int mn = pqMin.top(); // READ: menor -> O(1)
15 pqMin.pop();          // DELETE: O(log n)
16
17 // ! No existe acceso directo a elementos intermedios.
18 // ! priority_queue no tiene una operacion UPDATE directa.

```

5.7 Vector circular (Circular Buffer)

Idea: un arreglo de tamaño fijo (*cap*) que se comporta como una **cola doble**. En lugar de mover elementos cuando se inserta o elimina, se reutilizan las posiciones del arreglo de forma circular.

Se mantienen tres variables principales:

- *cap*: capacidad total del arreglo.
- *ini*: posición física donde comienza el primer elemento.
- *tam*: cantidad actual de elementos almacenados.

La posición lógica *i* no coincide necesariamente con la posición física del arreglo. Se obtiene mediante:

$$\text{posicionFisica}(i) = (\text{ini} + i) \% \text{cap}$$

Por ejemplo, con *cap* = 5 y *ini* = 3:

Índice lógico	0	1	2	3	4
Índice físico	3	4	0	1	2

Cuando se llega al final del arreglo, el módulo hace que la siguiente posición vuelva al comienzo. Por eso **no es necesario mover los elementos**.

¿Por qué usarlo? En un arreglo normal, insertar al frente requiere desplazar todos los elementos y cuesta $O(n)$. En un vector circular solo se mueve *ini*, por lo que cuesta $O(1)$.

Casos de uso:

- Colas de tamaño fijo.
- Buffers de audio, video o red.

- Ventanas deslizantes.
- Sistemas donde llegan y salen datos continuamente.
- Simulaciones de procesos en una cola.
- Mantener únicamente los últimos *k* elementos.

Si el buffer está lleno y se ejecuta **pushBack**, el nuevo elemento **reemplaza al más antiguo**. Esto permite mantener automáticamente una ventana de tamaño fijo.

Complejidad: insertar, eliminar, consultar extremos, acceder por posición y comprobar el tamaño son $O(1)$.

```

1 struct VectorCircular {
2     vector<int> buf;
3     int cap, ini = 0, tam = 0;
4
5     VectorCircular(int c) : buf(c), cap(c) {}
6
7     // CREATE: crea un buffer con capacidad fija.
8     // Los elementos inicialmente no estan ocupados.
9
10    bool empty() {
11        return tam == 0;
12    } // O(1)
13
14    bool full() {
15        return tam == cap;
16    } // O(1)
17
18    int size() {
19        return tam;
20    } // O(1)
21
22    int capacity() {
23        return cap;
24    } // O(1)
25
26    // Convierte una posicion logica en fisica.
27    int pos(int i) {
28        return (ini + i) % cap;
29    } // O(1)
30
31    // INSERT: agrega al final.
32    void pushBack(int x) {
33        buf[(ini + tam) % cap] = x;
34
35        if (tam < cap)
36            tam++;
37        else
38            ini = (ini + 1) % cap;
39    } // O(1)
40
41    // INSERT: agrega al inicio.
42    void pushFront(int x) {
43        ini = (ini - 1 + cap) % cap;
44        buf[ini] = x;
45
46        if (tam < cap)
47            tam++;
48    } // O(1)
49
50    // READ: consulta el primer elemento.
51    int front() {
52        return buf[ini];
53    } // O(1)
54
55    // READ: consulta el ultimo elemento.
56    int back() {
57        return buf[(ini + tam - 1) % cap];
58    } // O(1)
59
60    // READ: acceso por posicion logica.
61    int at(int i) {
62        return buf[(ini + i) % cap];
63    } // O(1)
64
65    // UPDATE: modifica un elemento.
66    void set(int i, int x) {
67        buf[(ini + i) % cap] = x;
68    } // O(1)
69
70    // DELETE: elimina el primer elemento.
71    int popFront() {
72        int v = buf[ini];
73        ini = (ini + 1) % cap;
74        tam--;
75        return v;
76    } // O(1)

```



```

77 // DELETE: elimina el ultimo elemento.
78 int popBack() {
79     tam--;
80     return buf[(ini + tam) % cap];
81 } // O(1)
82
83 // DELETE: elimina todos los elementos.
84 void clear() {
85     ini = 0;
86     tam = 0;
87 } // O(1)
88
89 // SEARCH: busca un elemento.
90 // A diferencia de las operaciones anteriores,
91 // hay que recorrer los elementos.
92 bool contains(int x) {
93     for (int i = 0; i < tam; i++)
94         if (at(i) == x)
95             return true;
96     return false;
97 } // O(n)
98
99 // READ: recorrer los elementos en orden logico.
100 void print() {
101     for (int i = 0; i < tam; i++)
102         cout << at(i) << " ";
103 } // O(n)
104 };
105
106

```

Ejemplo de funcionamiento:

```

1 VectorCircular v(5);
2
3 v.pushBack(10);
4 v.pushBack(20);
5 v.pushBack(30);
6 // [10, 20, 30, _, _]
7 // ini = 0, tam = 3
8
9 v.pushFront(5);
10 // [5, 10, 20, 30, _]
11 // ini cambia de 0 a 4
12
13 cout << v.front();
14 // 5
15
16 cout << v.back();
17 // 30
18
19 v.popFront();
20 // elimina 5
21 // [_, 10, 20, 30, _]
22
23 v.pushBack(40);
24 v.pushBack(50);
25 // [_, 10, 20, 30, 40, 50]
26 // El almacenamiento puede haber dado la vuelta.
27
28 v.pushBack(60);
29 // El buffer estaba lleno.
30 // Se agrega 60 y se elimina el mas antiguo.
31 // Resultado logico: [20, 30, 40, 50, 60]

```

Ventaja principal: el arreglo físico puede verse “desordenado”, pero los elementos siempre tienen un orden lógico determinado por `ini`. Por ejemplo:

```

1 // Memoria fisica:
2 // [40, 50, 20, 30, _]
3 //      ^
4 //      ini = 2
5
6 // Orden logico:
7 // 20 -> 30 -> 40 -> 50

```

No importa dónde estén físicamente los elementos: `at(0)` siempre representa el primero, `at(1)` el segundo, y así sucesivamente.

Diferencia con un vector normal:

Operación	vector	Circular
Agregar al final	$O(1)^*$	$O(1)$
Eliminar al final	$O(1)$	$O(1)$
Agregar al frente	$O(n)$	$O(1)$
Eliminar al frente	$O(n)$	$O(1)$
Acceso por índice	$O(1)$	$O(1)$
Buscar	$O(n)$	$O(n)$

*Amortizado.

Regla práctica: si necesitas una estructura de tamaño fijo donde los elementos entran y salen continuamente por los extremos, un vector circular evita los desplazamientos de un arreglo normal y mantiene las operaciones principales en $O(1)$.

6 Estructuras Hash/Tree

6.1 Conjuntos hash (HashSet)

Colección de elementos **sin duplicados** y sin orden garantizado. En C++ se implementa mediante `unordered_set`. Utiliza una tabla hash, por lo que las operaciones son $O(1)$ en promedio. Es útil cuando solo interesa saber si un elemento existe, sin necesidad de mantener los elementos ordenados.

Complejidad: insertar, buscar y borrar $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_set<int> s; // CREATE: conjunto vacio
2
3 s.insert(3); // INSERT: O(1) prom.
4 s.insert(7); // INSERT: O(1) prom.
5 s.insert(3); // no se agrega: ya existe
6
7 bool hay = s.count(3); // SEARCH: O(1) prom.
8 // hay = 1 si existe, 0 si no existe
9
10 auto it = s.find(7); // SEARCH: O(1) prom.
11 // devuelve iterador al elemento o s.end()
12
13 s.erase(3); // DELETE: O(1) prom.
14
15 int n = s.size(); // READ: cantidad -> O(1)
16 bool vacio = s.empty(); // READ: ¿esta vacio? -> O(1)
17
18 // Recorrer todos los elementos
19 for (int x : s)
20     cout << x << " ";
21 // O(n), pero el orden NO esta garantizado.
22
23 // ! No existe acceso por indice: s[0] no es valido.
24 // ! Los elementos repetidos se ignoran.
25 // ! El orden puede cambiar entre ejecuciones.

```

Cuándo usarlo:

- Saber rápidamente si un elemento ya apareció.
- Eliminar duplicados.
- Mantener un conjunto de elementos visitados.
- Detectar elementos repetidos en $O(n)$ promedio.
- BFS/DFS: guardar vértices visitados cuando sea conveniente.

6.2 Mapas / Diccionarios hash

Estructura que almacena pares **clave** → **valor**. En C++ se implementa mediante `unordered_map`. Cada clave identifica un valor y **no puede repetirse**. Si se asigna un nuevo valor a una clave existente, se reemplaza el anterior.

Complejidad: insertar, acceder, buscar y borrar por clave $O(1)$ promedio ($O(n)$ peor caso).


```

1 unordered_map<string,int> m; // CREATE: mapa vacio
2
3 m["a"] = 1; // INSERT: O(1) prom.
4 m["b"] = 2; // INSERT: O(1) prom.
5
6 int v = m["a"]; // READ: acceder -> O(1) prom.
7 // v = 1
8
9 m["a"] = 10; // UPDATE: O(1) prom.
10 // la clave "a" ahora tiene valor 10
11
12 m.erase("a"); // DELETE: O(1) prom.
13
14 bool existe = m.count("b"); // SEARCH: O(1) prom.
15 // existe = 1
16
17 auto it = m.find("b"); // SEARCH: O(1) prom.
18 if (it != m.end())
19     cout << it->second; // valor asociado a "b"
20
21 // Recorrer todas las parejas
22 for (auto [clave, valor] : m)
23     cout << clave << " " << valor;
24 // O(n), sin orden garantizado.

```

Importante: usar `m[clave]` para consultar una clave que no existe puede **crear esa clave** con su valor por defecto. Para comprobar existencia sin insertar, es preferible usar `count()` o `find()`.

```

1 unordered_map<string,int> m;
2
3 int x = m["a"];
4 // Si "a" no existia, se crea:
5 // "a" -> 0
6 // ! operator[] puede INSERTAR una clave al consultar.
7
8 // count(): comprueba si existe SIN insertar
9 bool existe = m.count("b");
10 // Devuelve 0 si no existe, 1 si existe.
11
12 // find(): busca la clave SIN insertar
13 auto it = m.find("b");
14
15 if (it != m.end())
16     cout << it->second; // valor asociado a "b"
17 else
18     cout << "No existe";
19
20 // ! count() solo indica si existe.
21 // ! find() permite acceder a la pareja clave -> valor.

```

Uso clásico: contar frecuencias.

```

1 unordered_map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++; // frecuencia de cada elemento
5
6 cout << freq[5];
7 // cantidad de veces que aparece 5

```

Cuándo usarlo:

- Contar frecuencias.
- Asociar identificadores con información.
- Buscar rápidamente un valor a partir de una clave.
- Problemas de conteo y frecuencia.
- Memorizar resultados de estados o funciones.

6.3 Conjuntos ordenados (TreeSet)

Conjunto de elementos **sin duplicados y ordenados**. En C++ se implementa mediante `set`, normalmente como un árbol balanceado.

A diferencia de `unordered_set`, los elementos siempre se mantienen ordenados según el comparador utilizado.

Complejidad: insertar, buscar y borrar $O(\log n)$. Consultar el menor o mayor elemento mediante `begin()` o `rbegin()` es $O(1)$.

```

1 set<int> s; // CREATE: conjunto vacio
2
3 s.insert(5); // INSERT: O(log n)
4 s.insert(1); // INSERT: O(log n)
5 s.insert(3); // INSERT: O(log n)
6 s.insert(3); // no se agrega: ya existe
7
8 int menor = *s.begin(); // READ: minimo -> O(1)
9 int mayor = *s.rbegin(); // READ: maximo -> O(1)
10
11 auto it = s.find(3); // SEARCH: O(log n)
12 if (it != s.end())
13     cout << *it;
14
15 s.erase(3); // DELETE: O(log n)
16
17 int n = s.size(); // READ: cantidad -> O(1)
18
19 // Recorrer en orden ascendente
20 for (int x : s)
21     cout << x << " ";
22 // O(n)

```

lower_bound y upper_bound:

```

1 set<int> s = {1, 3, 5, 7, 9};
2
3 auto it1 = s.lower_bound(5);
4 // primer elemento >= 5
5 // resultado: 5
6
7 auto it2 = s.upper_bound(5);
8 // primer elemento > 5
9 // resultado: 7

```

Esto permite resolver consultas de **piso y techo** rápidamente:

```

1 // FLOOR: mayor elemento <= x
2 auto it = s.upper_bound(x);
3
4 if (it != s.begin()) {
5     --it;
6     cout << *it;
7 }
8
9 // CEILING: menor elemento >= x
10 auto it2 = s.lower_bound(x);
11
12 if (it2 != s.end())
13     cout << *it2;

```

Cuándo usarlo:

- Cuando se necesitan elementos únicos y ordenados.
- Obtener mínimo o máximo dinámicamente.
- Encontrar el siguiente elemento mayor o igual.
- Encontrar el anterior elemento menor o igual.
- Mantener un conjunto mientras se insertan y eliminan elementos.

Diferencia clave: si solo necesitas saber si existe un elemento, `unordered_set` suele ser más rápido. Si necesitas consultar orden, `lower_bound`, `upper_bound`, `begin()` o `rbegin()`, `set` es la opción adecuada.

6.4 Mapas ordenados (TreeMap)

Mapa **clave** → **valor** cuyas claves se mantienen ordenadas. En C++ se implementa mediante `map`, usando un árbol balanceado.

Cada clave aparece una sola vez. Los valores pueden repetirse.

Complejidad: insertar, buscar, acceder y borrar por clave $O(\log n)$. `begin()` permite obtener la menor clave en $O(1)$.

```

1 map<string,int> m;           // CREATE: mapa vacio
2
3 m["b"] = 2;                 // INSERT: O(log n)
4 m["a"] = 1;                 // INSERT: O(log n)
5 m["c"] = 3;                 // INSERT: O(log n)
6
7 int v = m["a"];             // READ: O(log n)
8 // v = 1
9
10 m["a"] = 10;                // UPDATE: O(log n)
11
12 m.erase("b");               // DELETE: O(log n)
13
14 auto it = m.find("c");       // SEARCH: O(log n)
15 if (it != m.end())
16     cout << it->second;
17
18 // Primera pareja: menor clave
19 auto primera = m.begin();    // O(1)
20 cout << primera->first;
21
22 // Ultima pareja: mayor clave
23 auto ultima = prev(m.end()); // O(1) amortizado
24 cout << ultima->first;

```

Las claves se recorren ordenadas:

```

1 map<int,string> m;
2
3 m[30] = "C";
4 m[10] = "A";
5 m[20] = "B";
6
7 for (auto [clave, valor] : m)
8     cout << clave << " " << valor;
9
10 // Salida:
11 // 10 A
12 // 20 B
13 // 30 C

```

`lower_bound` y `upper_bound` también funcionan sobre las claves:

```

1 map<int,string> m;
2 m[10] = "A";
3 m[20] = "B";
4 m[30] = "C";
5
6 auto it = m.lower_bound(15);
7 // primera clave >= 15
8 // apunta a 20
9
10 auto it2 = m.upper_bound(20);
11 // primera clave > 20
12 // apunta a 30

```

Uso clásico: contar frecuencias ordenadas.

```

1 map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;
5
6 // Las claves aparecen en orden creciente.
7 for (auto [x, cantidad] : freq)
8     cout << x << ": " << cantidad << "\n";

```

Cuándo usarlo:

- Cuando se necesitan pares clave-valor ordenados.
- Contar frecuencias y procesarlas en orden.
- Encontrar claves inmediatamente mayores o menores.
- Procesar eventos o valores manteniendo un orden dinámico.
- Cuando se necesitan operaciones de árbol como `lower_bound` y `upper_bound`.

Comparación rápida

Estructura	Duplicados	Orden	Buscar
<code>unordered_set</code>	No	No	$O(1)$ prom.
<code>unordered_map</code>	Claves: No	No	$O(1)$ prom.
<code>set</code>	No	Sí	$O(\log n)$
<code>map</code>	Claves: No	Sí	$O(\log n)$

Regla práctica:

- `unordered_set` → ¿Ya existe este elemento?
- `unordered_map` → ¿Qué valor corresponde a esta clave?
- `set` → Elementos únicos + ordenados.
- `map` → Claves → valores + claves ordenadas.

7 Trees

8 Range Queries

8.1 Casos de uso

Cuando el problema te pide:	Entonces usa:	Complejidad
Suma de rango sobre un arreglo que nunca cambia , con muchas consultas (ej. "suma de ventas entre el día l y r ")	Prefix Sums	Build $O(n)$ Query $O(1)$
Suma de rango con actualizaciones puntuales frecuentes (ej. "suma el stock de la posición i , luego consulta un rango")	Fenwick Tree (BIT)	Build $O(n)$ Query/Update $O(\log n)$
Sumar v a todo un rango $[l, r]$ y también consultar la suma de cualquier rango	FenwickRangeUpdate (doble BIT)	Query/Update $O(\log n)$
Range update + range query con operaciones más complejas: min/max en rango, asignar un valor a todo un rango, contar ceros, etc.	Segment Tree + Lazy Propagation	Query/Update $O(\log n)$
Mínimo, máximo o gcd de cualquier rango sobre un arreglo estático (ej. RMQ clásico, LCA) Query $O(1)$	Sparse Table	Build $O(n \log n)$
Mínimo o máximo de rango cuando el arreglo sí cambia con el tiempo	Segment Tree	Query/Update $O(\log n)$
Consultas 2D: cuántos puntos o cuánta suma hay dentro de un rectángulo $[x_1, x_2] \times [y_1, y_2]$	Fenwick 2D	Query/Update $O(\log n \log m)$
Encontrar el k -ésimo elemento, o "menor índice cuya suma acumulada alcanza X " (frecuencias)	Fenwick con <code>lowerBound</code>	$O(\log n)$
Muchas queries de rango offline (las conoces todas de antemano) sobre un arreglo estático o con pocos updates	Mo's Algorithm	$O((n+q)\sqrt{n})$
LCA entre dos nodos, o mínimo en rango sobre un recorrido Euler de un árbol (valores consecutivos difieren en ± 1)	RMQ ± 1 (Euler Tour + Sparse Table)	Build $O(n)$ Query $O(1)$
Cuántos elementos en el rango $[l, r]$ están dentro de un rango de valores $[a, b]$, o cuál es el k -ésimo menor en $[l, r]$	Wavelet Tree	Query $O(\log n)$
Queries o updates sobre caminos o subárboles de un árbol (ej. "suma el camino de u a v ")	Heavy-Light Decomposition (+ Segment/Fenwick)	Query/Update $O(\log^2 n)$
Necesitas algo simple de implementar bajo presión, con balance razonable entre update y query, sin memorizar mucha teoría	Sqrt Decomposition	Query/Update $O(\sqrt{n})$

Table 2: Guía rápida: qué estructura de range queries usar según el problema, con su complejidad.

8.2 Prefix Sums

8.3 Suma de Euler Totient (ϕ) en un rango

Idea: $\phi(n)$ cuenta cuántos enteros en $[1, n]$ son coprimos con n . Calcular ϕ de un número es teoría de números ($O(\sqrt{n})$ factorizando); pero cuando el problema pide la **suma de $\phi(i)$ para i en un rango $[l, r]$** , con muchas queries, el patrón es en dos fases: (1) precalculas el arreglo de ϕ una sola vez con una criba, y (2) respondes las queries de rango con las estructuras de esta sección (Prefix Sums o Fenwick) sobre ese arreglo ya calculado. La fase (1) es teoría de números; la fase (2) es exactamente Range Queries.

¿Por qué usarlo? Factorizar cada número del rango uno por uno para responder cada query sería $O(q\sqrt{n})$ y no escala. Precalcular ϕ una vez y sumar con prefijos reduce cada query a $O(1)$, sin importar cuántas se hagan.

Casos de uso:

- Suma de $\phi(i)$ (u otra función aritmética calculable por criba, como μ o el número de divisores) en un rango $[l, r]$, con $l, r \leq N$ y N manejable ($\leq 10^7$ aprox.).
- El mismo problema pero con $[L, R]$ muy grandes ($R \leq 10^{12}$) y $R - L$ pequeño: usa criba segmentada en vez de criba desde 1.
- Cualquier variante donde ϕ no cambia entre queries (arreglo estático) $\Rightarrow$ Prefix Sums. Si además necesitas modificar valores puntuales entre queries, usa Fenwick.

Complejidad: criba lineal $O(N)$, criba segmentada $O((R - L + 1) \log \log R + \sqrt{R})$; en ambos casos, una vez armado el arreglo, sumar un rango es $O(1)$ con Prefix Sums o $O(\log N)$ con Fenwick si hace falta actualizar.

Criba lineal de ϕ + Prefix Sums (rango $[1, N]$)

```

1 struct PhiRangeSum {
2     vector<long long> phi, pref;
3     int n;
4
5     // CREATE: calcula phi[1..n] con criba lineal y sus prefijos.
6     // phi[i] = cantidad de enteros en [1, i] coprimos con i.
7     PhiRangeSum(int n) : n(n) {
8         phi.assign(n + 1, 0);
9         pref.assign(n + 1, 0);
10
11         vector<int> primos;
12         vector<bool> compuesto(n + 1, false);
13         phi[1] = 1;
14
15         for (int i = 2; i <= n; i++) {
16             if (!compuesto[i]) {
17                 primos.push_back(i);
18                 phi[i] = i - 1;
19             }
20
21             for (int p : primos) {
22                 if ((long long)i * p > n) break;
23
24                 compuesto[i * p] = true;
25
26                 if (i % p == 0) {
27                     phi[i * p] = phi[i] * p;
28                     break;
29                 } else {
30                     phi[i * p] = phi[i] * (p - 1);
31                 }
32             }
33         }
34
35         for (int i = 1; i <= n; i++)

```

```

36         pref[i] = pref[i - 1] + phi[i];
37     } // O(n)
38
39     // READ: suma de phi(i) para i en [l, r].
40     long long rango(int l, int r) {
41         if (l > r) return 0;
42         return pref[r] - pref[l - 1];
43     } // O(1)
44
45     // READ: valor individual de phi(i).
46     long long get(int i) {
47         return phi[i];
48     } // O(1)
49 };

```

Ejemplo de funcionamiento:

```

1 PhiRangeSum pr(10);
2 // phi(1..10) = [1, 1, 2, 2, 4, 2, 6, 4, 6, 4]
3
4 cout << pr.get(7);
5 // phi(7) = 6 (7 es primo)
6
7 cout << pr.rango(1, 10);
8 // 1+1+2+2+4+2+6+4+6+4 = 32
9
10 cout << pr.rango(4, 8);
11 // 2+4+2+6+4 = 18

```

Criba segmentada de ϕ (rango $[L, R]$ grande)

Idea: si R es muy grande (10^{12}) pero $R - L$ es manejable, no puedes cribar desde 1. En vez de eso, precalculas los primos hasta $\sqrt{R}$ con una criba normal, y para cada número del segmento $[L, R]$ le vas quitando sus factores primos pequeños, aplicando la fórmula $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$. Si al final queda un residuo > 1 , es un único primo grande (mayor a $\sqrt{R}$) que también se aplica a la fórmula.

```

1 struct PhiSegmentedRangeSum {
2     vector<long long> phi, pref;
3     long long L, R;
4
5     // CREATE: calcula phi(i) para cada i en [L, R] factorizando
6     // con los primos hasta sqrt(R), y sus prefijos sobre el segmento.
7
8     PhiSegmentedRangeSum(long long L, long long R) : L(L), R(R) {
9         int lim = (int)sqrt((double)R) + 1;
10
11         vector<int> primos;
12         vector<bool> compuesto(lim + 1, false);
13         for (int i = 2; i <= lim; i++) {
14             if (!compuesto[i]) primos.push_back(i);
15             for (int p : primos) {
16                 if ((long long)i * p > lim) break;
17                 compuesto[i * p] = true;
18                 if (i % p == 0) break;
19             }
20         } // O(sqrt(R) log
21
22         int sz = R - L + 1;
23         vector<long long> num(sz);
24         phi.assign(sz, 0);
25
26         for (int i = 0; i < sz; i++) {
27             num[i] = L + i;
28             phi[i] = L + i;
29         }
30
31         for (int p : primos) {
32             long long start = ((L + p - 1) / p) * p;
33
34             for (long long j = start; j <= R; j += p) {
35                 int idx = j - L;
36                 phi[idx] -= phi[idx] / p;
37
38                 while (num[idx] % p == 0)
39                     num[idx] /= p;
40             }
41
42             // lo que quede > 1 es un primo grande (> sqrt(R)).
43             for (int i = 0; i < sz; i++)
44                 if (num[i] > 1)
45                     phi[i] -= phi[i] / num[i];

```

```

46     pref.assign(sz + 1, 0);
47     for (int i = 0; i < sz; i++)
48         pref[i + 1] = pref[i] + phi[i];
49 }
50 // log R + sqrt(R)
51
52 // READ: suma de phi(i) para i en [l, r], con L <= l <= r <=
53 // R.
54 long long rango(long long l, long long r) {
55     return pref[r - L + 1] - pref[l - L];
56 }
57 // O(1)
58
59 // READ: valor individual de phi(i), con L <= i <= R.
60 long long get(long long i) {
61     return phi[i - L];
62 }
63 // O(1)

```

Ejemplo de funcionamiento:

```

1 PhiSegmentedRangeSum ps(95, 105);
2 // calcula phi(95), phi(96), ..., phi(105)
3 // sin cribar desde 1
4
5 cout << ps.get(100);
6 // phi(100) = 40
7
8 cout << ps.rango(95, 105);
9 // suma de phi(95) a phi(105)
10
11 cout << ps.rango(100, 102);
12 // phi(100) + phi(101) + phi(102)

```

Cuál variante usar:

Situación	Estructura	Complejidad
$[1, N]$, N chico/mediano	PhiRangeSum	$O(N)$
$[L, R]$ grande, $R - L$ chico	PhiSegmentedRangeSum	$O((R-L) \log \log R + \sqrt{R})$
ϕ cambia entre queries	Fenwick sobre ϕ	$O(\log N)$

Table 3: Variantes para la suma de ϕ .

Regla práctica: el cálculo de ϕ en sí (la criba, lineal o segmentada) es teoría de números y no cambia según la query. Una vez que tienes el arreglo de ϕ listo, tratarlo como *cualquier otro arreglo*: si no cambia, Prefix Sums; si necesitas actualizarlo puntualmente entre queries, Fenwick. No hace falta una estructura especial para ϕ más allá de construir bien el arreglo inicial.

8.4 Árbol de Fenwick (BIT)

Idea: un arreglo normal da suma de prefijo en $O(1)$ pero actualizar un elemento cuesta $O(n)$ (o al revés si guardas prefijos). El Fenwick equilibra ambas a $O(\log n)$ guardando *sumas parciales inteligentes*: la posición i almacena la suma de un bloque de tamaño $i \& (-i)$ (su bit menos significativo) que termina en i . Para consultar un prefijo saltas hacia atrás quitando el bit bajo ($i -= i \& (-i)$); para actualizar subes sumándolo ($i += i \& (-i)$). Cada camino toca a lo más $\log n$ posiciones (un bit por paso). Es 1-indexado.

¿Por qué usarlo? Es mucho más simple y liviano que un Segment Tree cuando solo necesitas sumas (o cualquier operación invertible) de prefijo/rango con actualizaciones puntuales. Menos memoria, menos código y una constante más pequeña.

Casos de uso:

- Suma de prefijo / suma de rango con updates frecuentes.
- Conteo de inversiones en un arreglo.
- Frecuencias acumuladas y búsqueda del k -ésimo elemento.

- Range update + range query cuando no hace falta un Segment Tree con Lazy Propagation.
- Consultas 2D: conteo/suma de puntos dentro de un rectángulo.

Complejidad: construir $O(n)$, update puntual $O(\log n)$, query de prefijo/rango $O(\log n)$, range update + range query $O(\log n)$ y versión 2D $O(\log n \cdot \log m)$.

BIT clásico (Point Update, Range Query)

```

1 struct FenwickTree {
2     vector<long long> t;
3     int n;
4
5     // CREATE: reserva un arbol de tamaño n (1-indexado).
6     FenwickTree(int n) : t(n + 1, 0), n(n) {}
7
8     // CREATE: construye el arbol a partir de un arreglo a[0..n-1]
9     // propagando cada valor a su "padre" una sola vez, en vez de
10    // hacer n updates (que serian O(n log n)).
11    void build(vector<long long>& a) {
12        for (int i = 1; i <= n; i++) {
13            t[i] += a[i - 1];
14
15            int j = i + (i & (-i));
16
17            if (j <= n)
18                t[j] += t[i];
19        }
20    } // O(n)
21
22    // UPDATE: suma v al elemento en la posición i.
23    void update(int i, long long v) {
24        for (; i <= n; i += i & (-i))
25            t[i] += v;
26    } // O(log n)
27
28    // UPDATE: fija el valor absoluto de la posición i en x.
29    void set(int i, long long x) {
30        update(i, x - get(i));
31    } // O(log n)
32
33    // READ: suma de prefijo [1..i].
34    long long query(int i) {
35        long long s = 0;
36
37        for (; i > 0; i -= i & (-i))
38            s += t[i];
39
40        return s;
41    } // O(log n)
42
43    // READ: suma del rango [l, r].
44    long long rango(int l, int r) {
45        if (l > r)
46            return 0;
47
48        return query(r) - query(l - 1);
49    } // O(log n)
50
51    // READ: valor puntual en la posición i.
52    long long get(int i) {
53        return rango(i, i);
54    } // O(log n)
55
56    // SEARCH: menor índice i tal que query(i) >= objetivo.
57    // Usa binary lifting sobre el arbol.
58    // Requiere que todos los valores sean >= 0.
59    int lowerBound(long long objetivo) {
60        int pos = 0;
61        long long ac = 0;
62
63        int LOG = 0;
64
65        while ((1 << LOG) <= n)
66            LOG++;
67
68        for (int k = LOG - 1; k >= 0; k--) {
69            int nx = pos + (1 << k);
70
71            if (nx <= n && ac + t[nx] < objetivo) {
72                pos = nx;
73                ac += t[nx];
74            }
75        }
76    }

```

```

77     return pos + 1;
78 }                                     // O(log n)
79
80 // DELETE: reinicia el arbol a ceros.
81 void clear() {
82     fill(t.begin(), t.end(), 0);
83 }                                     // O(n)
84 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 FenwickTree bit(5);
4 bit.build(a);
5
6 // arreglo logico:
7 // [5, 2, 9, 1, 7]
8 // posiciones 1..5
9
10 cout << bit.query(3);
11 // 5 + 2 + 9 = 16
12
13 cout << bit.rango(2, 4);
14 // 2 + 9 + 1 = 12
15
16 bit.update(2, 3);
17
18 // arreglo logico:
19 // [5, 5, 9, 1, 7]
20
21 cout << bit.get(2);
22 // 5
23
24 bit.set(4, 10);
25
26 // arreglo logico:
27 // [5, 5, 9, 10, 7]
28
29 cout << bit.lowerBound(15);
30 // menor indice cuyo prefijo alcanza 15
31 // -> indice 3 (5 + 5 + 9 = 19 >= 15)

```

BIT con Range Update + Range Query (Doble BIT)

Idea: con *dos* Fenwick y el truco de diferencias se puede sumar v a todo un rango $[l, r]$ en $O(\log n)$ y también consultar la suma de cualquier rango en $O(\log n)$. Esto permite resolver simultáneamente actualizaciones y consultas de rango.

```

1 struct FenwickRangeUpdate {
2     FenwickTree b1, b2;
3     int n;
4
5     // CREATE: reserva dos arboles auxiliares de tamaño n.
6     FenwickRangeUpdate(int n) : b1(n), b2(n), n(n) {}
7
8     // UPDATE: suma v a todos los elementos del rango [l, r].
9     void rangoUpdate(int l, int r, long long v) {
10         b1.update(l, v);
11         b1.update(r + 1, -v);
12
13         b2.update(l, v * (l - 1));
14         b2.update(r + 1, -v * r);
15     }                                     // O(log n)
16
17     // READ: suma de prefijo [1..i].
18     long long prefijo(int i) {
19         return b1.query(i) * i - b2.query(i);
20     }                                     // O(log n)
21
22     // READ: suma del rango [l, r].
23     long long rango(int l, int r) {
24         if (l > r)
25             return 0;
26
27         return prefijo(r) - prefijo(l - 1);
28     }                                     // O(log n)
29
30     // READ: valor puntual en la posición i.
31     long long get(int i) {
32         return rango(i, i);
33     }                                     // O(log n)
34 };

```

Ejemplo de funcionamiento:

```

1 FenwickRangeUpdate f(5);
2 // arreglo logico inicial: [0, 0, 0, 0, 0]
3
4 f.rangoUpdate(2, 4, 3);
5 // arreglo logico: [0, 3, 3, 3, 0]
6
7 cout << f.rango(1, 5);
8 // 0+3+3+3+0 = 9
9
10 f.rangoUpdate(1, 3, 2);
11 // arreglo logico: [2, 5, 5, 3, 0]
12
13 cout << f.get(2);
14 // 5

```

BIT 2D (point update, sum de submatriz)

Idea: un BIT dentro de otro BIT. Cada update/query en x dispara un update/query completo en y , dando $O(\log n \cdot \log m)$ en vez de $O(nm)$ por consulta.

```

1 struct Fenwick2D {
2     vector<vector<long long>> t;
3     int n, m;
4
5     // CREATE: reserva una matriz de arboles de tamaño n x m.
6     Fenwick2D(int n, int m) : t(n + 1, vector<long long>(m + 1, 0)),
7                               n(n), m(m) {}
8
9     // UPDATE: suma v a la celda (x, y).
10    void update(int x, int y, long long v) {
11        for (int i = x; i <= n; i += i & (-i))
12            for (int j = y; j <= m; j += j & (-j))
13                t[i][j] += v;
14    }                                     // O(log n * log m)
15
16    // READ: suma de la submatriz [1..x] x [1..y].
17    long long query(int x, int y) {
18        long long s = 0;
19        for (int i = x; i > 0; i -= i & (-i))
20            for (int j = y; j > 0; j -= j & (-j))
21                s += t[i][j];
22        return s;
23    }                                     // O(log n * log m)
24
25    // READ: suma del rectangulo [x1..x2] x [y1..y2].
26    long long rango(int x1, int y1, int x2, int y2) {
27        return query(x2, y2) - query(x1 - 1, y2)
28            - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
29    }                                     // O(log n * log m)
30 };

```

Ejemplo de funcionamiento:

```

1 Fenwick2D f(4, 4);
2
3 f.update(1, 1, 5);
4 f.update(2, 3, 7);
5 f.update(4, 4, 2);
6
7 cout << f.rango(1, 1, 2, 3);
8 // 5 + 7 = 12
9
10 cout << f.rango(3, 3, 4, 4);
11 // 2

```

Diferencia con un arreglo de prefijos simple:

Estructura	Construir	Query	Update
Prefijos simples	$O(n)$	$O(1)$	$O(n)$
FenwickTree	$O(n)$	$O(\log n)$	$O(\log n)$
FenwickRangeUpdate	$O(n)$	$O(\log n)$	$O(\log n)^*$
Fenwick2D	$O(nm)$	$O(\log n \log m)$	$O(\log n \log m)$

Table 4: Comparación de complejidades: Fenwick vs. prefijos simples.

*Range update + range query, con la variante de doble BIT.

Regla práctica: si el arreglo cambia con frecuencia y solo necesitas sumas (o cualquier operación invertible)

de prefijo o rango, el Fenwick da el mejor balance entre simplicidad de código y velocidad. Si necesitas min/max, operaciones no invertibles, o updates de rango con lazy propagation general, usa Segment Tree en su lugar.

8.5 Sqrt Decomposition

8.6 Sparse Table (RMQ)

Idea: responde consultas de rango en $O(1)$ sobre un arreglo que **no cambia**, precalculando $sp[k][i]$ = el resultado de combinar el bloque que empieza en i y tiene largo 2^k . La clave está en la consulta: para el rango $[l, r]$ tomas $k = \lfloor \log_2(r - l + 1) \rfloor$ y lo cubres con *dos* bloques de largo 2^k —uno pegado a l y otro pegado a r — que **se solapan**. Si la operación es *idempotente* ($f(x, x) = x$, como min, max o gcd), contar el solape dos veces no cambia el resultado, así que la consulta son solo dos lecturas y una combinación. Para operaciones **no idempotentes** (como suma o xor) hace falta la variante *Disjoint Sparse Table*, que evita el solape por construcción.

¿Por qué usarlo? Es la estructura más rápida posible para consultas de rango ($O(1)$ por query) cuando el arreglo es estático. A cambio de esa velocidad, no soporta actualizaciones: reconstruir cuesta $O(n \log n)$, así que si el arreglo cambia, conviene un Segment Tree.

Casos de uso:

- RMQ clásico: mínimo o máximo de cualquier rango, arreglo estático.
- gcd o AND/OR de rango sobre un arreglo estático.
- LCA vía Euler Tour + RMQ ± 1 .
- Suma, xor o cualquier operación no idempotente de rango estático, en $O(1)$, usando Disjoint Sparse Table.
- Cualquier problema con muchísimas queries offline sobre datos que nunca se modifican.

Complejidad: construir $O(n \log n)$, consulta $O(1)$. (Si el arreglo *cambia*, usa Segment Tree: $O(\log n)$ por operación.)

Sparse Table clásico (operaciones idempotentes: min, max, gcd)

```
1 struct SparseTable {
2     vector<vector<long long>>> sp;
3     vector<int> lg;
4     int n;
5
6     // CREATE: precalcula la tabla a partir de un arreglo a[0..n-1].
7     // sp[k][i] = mínimo del bloque [i, i + 2^k - 1].
8     SparseTable(vector<long long>& a) {
9         n = a.size();
10
11         lg.assign(n + 1, 0);
12         for (int i = 2; i <= n; i++)
13             lg[i] = lg[i / 2] + 1;
14
15         int K = lg[n] + 1;
16         sp.assign(K, vector<long long>(n));
17         sp[0] = a;
18
19         for (int k = 1; k < K; k++)
20             for (int i = 0; i + (1 << k) <= n; i++)
21                 sp[k][i] = min(sp[k - 1][i],
22                               sp[k - 1][i + (1 << (k - 1))]);
23     }
24 }
```

```
23 } // O(n log n)
24
25 // READ: mínimo del rango [l, r] (0-indexado, inclusivo).
26 // Cubre [l, r] con dos bloques de largo 2^k que se solapan;
27 // como min es idempotente, el solape no afecta el resultado.
28 long long query(int l, int r) {
29     int k = lg[r - l + 1];
30     return min(sp[k][l], sp[k][r - (1 << k) + 1]);
31 } // O(1)
32 }
```

Ejemplo de funcionamiento:

```
1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SparseTable st(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << st.query(0, 2);
7 // min(5, 2, 9) = 2
8
9 cout << st.query(2, 5);
10 // min(9, 1, 7, 3) = 1
11
12 cout << st.query(4, 4);
13 // min(7) = 7
```

Nota: para max o gcd solo cambia $\min(\dots)$ por $\max(\dots)$ o $_\gcd(\dots)$ en el constructor y en query. Si necesitas saber *qué índice* alcanza el mínimo (no solo el valor), guarda índices en vez de valores y compara $a[sp[k][i]]$ en lugar de $sp[k][i]$.

Disjoint Sparse Table (operaciones no idempotentes: suma, xor)

Idea: en vez de dos bloques que se solapan, cada nivel k particiona el arreglo en bloques de largo 2^{k+1} y precalcula, para cada bloque, un prefijo hacia la izquierda desde su punto medio y un prefijo hacia la derecha desde ese mismo punto medio. Toda consulta $[l, r]$ cae exactamente en un nivel donde l y r quedan en mitades distintas del mismo bloque (el nivel lo da el bit más alto donde difieren l y r), así que se combina sin solapar y sirve para operaciones **no idempotentes** como la suma.

```
1 struct DisjointSparseTable {
2     vector<vector<long long>>> pre;
3     vector<long long> a;
4     int n, LOG;
5
6     // CREATE: precalcula prefijos izquierda/derecha por nivel,
7     // partiendo el arreglo en bloques de tamaño 2^(k+1).
8     DisjointSparseTable(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        LOG = 1;
12        while ((1 << LOG) < n) LOG++;
13        LOG++;
14
15        pre.assign(LOG, vector<long long>(n));
16
17        for (int k = 0; k < LOG; k++) {
18            int block = 1 << (k + 1);
19
20            for (int start = 0; start < n; start += block) {
21                int mid = min(start + block / 2, n);
22                int end = min(start + block, n);
23
24                if (mid > n) continue;
25
26                // prefijo hacia la izquierda desde mid-1
27                if (mid - 1 >= start) {
28                    pre[k][mid - 1] = a[mid - 1];
29                    for (int i = mid - 2; i >= start; i--)
30                        pre[k][i] = a[i] + pre[k][i + 1];
31                }
32
33                // prefijo hacia la derecha desde mid
34                if (mid < end) {
35                    pre[k][mid] = a[mid];
36                    for (int i = mid + 1; i < end; i++)
37                        pre[k][i] = pre[k][i - 1] + a[i];
38                }
39            }
40        }
41    }
```

```

40     }
41 } // O(n log n)
42
43 // READ: suma del rango [l, r] (0-indexado, inclusivo).
44 // El nivel k lo da el bit mas alto donde l y r difieren:
45 // ahi es donde el bloque se parte en dos mitades disjuntas.
46 long long query(int l, int r) {
47     if (l == r) return a[l];
48
49     int k = 31 - __builtin_clz(l ^ r);
50     return pre[k][l] + pre[k][r];
51 } // O(1)
52 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 DisjointSparseTable dst(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << dst.query(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << dst.query(2, 5);
10 // 9 + 1 + 7 + 3 = 20
11
12 cout << dst.query(1, 1);
13 // 2

```

Diferencia entre las dos variantes:

Estructura	Construir	Query	Updates
Sparse Table (min/max/gcd)	$O(n \log n)$	$O(1)$	No
Disjoint Sparse Table (suma/xor)	$O(n \log n)$	$O(1)$	No
Segment Tree	$O(n)$	$O(\log n)$	Sí, $O(\log n)$

Table 5: Sparse Table vs. Disjoint Sparse Table vs. Segment Tree.

Regla práctica: si el arreglo **no cambia**, usa Sparse Table para min/max/gcd, o Disjoint Sparse Table si necesitas suma, xor, o cualquier operación no idempotente. En cuanto el arreglo **cambia** —aunque sea una sola actualización— reconstruir en $O(n \log n)$ deja de valer la pena y conviene un Segment Tree.

8.7 Segment Tree

8.8 RMQ (Range Minimum Query)

8.9 Mo's Algorithm

8.10 Wavelet Tree

8.11 Heavy-Light Decomposition

Parte III Grafos y Árboles

9 Grafos (Graphs)

Contenido en desarrollo.

10 Árboles (Trees)

Contenido en desarrollo.

11 Disjoint Set Union (DSU)

Contenido en desarrollo.

Parte IV Ordenamiento y Búsqueda

12 Ordenamiento y Búsqueda (Sorting & Searching)

Varias búsquedas lineales y sub-lineales. (La búsqueda binaria tiene su propia sección.)

12.1 Número faltante (Missing Number)

Idea: los números $0, 1, \dots, n$ deberían sumar $\frac{n(n+1)}{2}$ (fórmula de Gauss). Si falta uno, la suma real queda corta *exactamente* en ese valor, así que el faltante es (suma esperada – suma real). Basta un recorrido para acumular la suma. Alternativa sin riesgo de overflow: hacer XOR de $0..n$ y de todos los elementos; lo que quede es el faltante. **Complejidad:** $O(n)$.

```

1 long long esp = (long long)n*(n+1)/2; // suma 0..n
2 long long falta = esp - accumulate(a.begin(), a.end(), 0LL);

```

12.2 Máximo / Mínimo (Array Max/Min)

Idea: recorres el arreglo una sola vez llevando el menor y el mayor vistos hasta ahora; comparás cada elemento contra ambos y actualizas. No requiere ordenar (ordenar sería $O(n \log n)$, peor). Con un recorrido consigues los dos a la vez. **Complejidad:** $O(n)$.

```

1 int mn = *min_element(a.begin(), a.end());
2 int mx = *max_element(a.begin(), a.end());

```

12.3 Par de suma absoluta mínima

Idea: buscamos el par cuya suma esté *más cerca de cero*. Tras ordenar, ponemos un puntero en cada extremo. La suma de los extremos te dice hacia dónde moverte: si es negativa, para acercarla a 0 hay que subir el menor (`lo++`); si es positiva, hay que bajar el mayor (`hi--`). Cada movimiento descarta el candidato menos prometedor, y en el camino guardas el $|suma|$ más pequeño visto. Los dos punteros recorren el arreglo una vez ($O(n)$); domina el sort. **Complejidad:** $O(n \log n)$.

```

1 sort(a.begin(), a.end());
2 int lo=0, hi=a.size()-1; long long mej=LLONG_MAX;
3 while(lo<hi){ long long s=(long long)a[lo]+a[hi];
4     mej=min(mej, llabs(s));
5     if(s<0) lo++; else hi--; }

```

12.4 Par con diferencia dada (Difference Pair)

Idea: ¿existe un par cuya diferencia sea exactamente d ? Tras ordenar, dos punteros avanzan hacia adelante: si la diferencia actual $a_j - a_i$ es menor que d , adelantas el puntero lejano (j) para agrandarla; si es mayor, adelantas el cercano (i) para achicarla; si es igual, encontraste el par. Como el arreglo está ordenado, la diferencia crece con j y decrece con i , así que el barrido nunca retrocede. **Complejidad:** $O(n \log n)$ (domina el sort).

```

1 sort(a.begin(), a.end()); int i=0, j=1, n=a.size();
2 while(i<n && j<n){
3     if(i!=j && a[j]-a[i]==d) return true;
4     if(a[j]-a[i]<d) j++; else i++; }

```


12.5 Búsqueda ternaria (Ternary Search)

Idea: sirve para hallar el pico de una función **unimodal** (sube y luego baja, con un solo máximo — no para buscar un valor en un arreglo cualquiera). Partes el rango en tres con dos puntos $m_1 < m_2$ y comparas: si $f(m_1) < f(m_2)$, el máximo no puede estar a la izquierda de m_1 , así que descartas ese tercio; en caso contrario descartas el tercio derecho. Cada paso elimina $\sim \frac{1}{3}$ del rango, por eso converge en logarítmico. (Para un mínimo, invierte la comparación.) **Complejidad:** $O(\log n)$ — NO $O(n \log n)$.

```
1 while(hi-lo>2){ long long m1=lo+(hi-lo)/3, m2=hi-(hi-lo)/3;
2   if(f(m1)<f(m2)) lo=m1+1; else hi=m2-1; }
3 // revisa [lo,hi] y devuelve el x que maximiza f
```

12.6 Búsqueda de Fibonacci (Fibonacci Search)

Idea: misma estrategia que la binaria (descartar mitades de un arreglo ordenado), pero en vez de partir por el punto medio elige el punto de comparación usando números de Fibonacci consecutivos. Mantiene tres Fibonacci y en cada paso mira la posición $off+f2$; según el elemento sea menor o mayor que el objetivo, retrocede uno o dos pasos en la secuencia. Ventaja: solo usa *sumas y restas*, nunca división, lo que la hace útil en hardware sin división rápida o para saltar por posiciones amigables con la caché. **Complejidad:** $O(\log n)$.

```
1 int f2=0,f1=1,fb=f1+f2;
2 while(fb<n){ f2=f1; f1=fb; fb=f1+f2; }
3 int off=-1;
4 while(fb>1){ int i=min(off+f2,n-1);
5   if(a[i]<x){ fb=f1; f1=f2; f2=fb-f1; off=i; }
6   else if(a[i]>x){ fb=f2; f1=f1-f2; f2=fb-f1; }
7   else return i; }
8 if(f1==1 && off+1<n && a[off+1]==x) return off+1;
```

(En Java es idéntico al C++ con `a.length`.)

12.7 Búsqueda exponencial (Exponential Search)

Idea: dos fases sobre un arreglo **ordenado**. Primero *acotas* el objetivo doblando el índice (1, 2, 4, 8, ...) hasta que $a[i]$ lo supera; en ese momento sabes que el objetivo está entre $i/2$ e i . Segundo, haces una binaria normal dentro de ese bloque. Como el bloque encontrado tiene tamaño proporcional a la posición del objetivo, es ideal cuando el arreglo es enorme o de tamaño desconocido y el objetivo está cerca del inicio. **Complejidad:** $O(\log n)$.

```
1 if(a[0]==x) return 0;
2 int i=1; while(i<n && a[i]<=x) i*=2; // dobla el rango
3 // luego binaria en [i/2, min(i,n-1)]
```

12.8 Búsqueda por saltos (Jump Search)

Idea: en vez de revisar elemento por elemento, avanzas en bloques de tamaño $\sqrt{n}$ mirando solo el *último* de cada bloque; cuando ese último ya supera al objetivo, sabes que (si existe) está en el bloque anterior, así que lo barres linealmente. El balance $\sqrt{n}$ saltos + $\sqrt{n}$ del barrido minimiza el total. Punto medio entre la lineal ($O(n)$) y la binaria ($O(\log n)$), útil cuando saltar hacia atrás es costoso. **Complejidad:** $O(\sqrt{n})$.

```
1 int paso=max(1,(int)sqrt((double)n)), prev=0;
2 while(prev<n && a[min(prev+paso,n)-1]<=x) prev+=paso;
3 for(int i=prev; i<min(prev+paso,n); i++)
4   if(a[i]==x) return i;
```

12.9 Heap Sort

Idea: un *heap* binario se representa en el mismo arreglo: el hijo izquierdo de i es $2i+1$ y el derecho $2i+2$; en un *max-heap* todo padre es $\geq$ sus hijos, así que el mayor está en la raíz. Heap Sort tiene dos fases. **(1) Construir el heap:** recorres de abajo hacia arriba aplicando **bajar** (sift-down) a cada padre. Aunque parezca $O(n \log n)$, es $O(n)$: casi todos los nodos están cerca de las hojas y se hunden poco (la suma $\sum n/2^h \cdot h$ converge a $2n$). **(2) Ordenar:** repetidamente intercambia la raíz (el mayor) con el último elemento activo — quedando ya ordenado al final — y “hundes” la nueva raíz para restaurar el heap; cada una de las n extracciones cuesta $O(\log n)$. In-place, sin memoria extra, pero **no estable**. **Complejidad:** $O(n \log n)$ en todos los casos.

```
1 void bajar(vector<int>&a,int n,int i){ // hunde: O(log n)
2   while(true){ int m=i,l=2*i+1,r=2*i+2;
3     if(l<n&&a[l]>a[m]) m=l;
4     if(r<n&&a[r]>a[m]) m=r;
5     if(m==i) break; swap(a[i],a[m]); i=m; } }
6 void heapSort(vector<int>&a){ int n=a.size();
7   for(int i=n/2-1;i>0;i--) bajar(a,n,i); // heap: O(n)
8   for(int i=n-1;i>0;i--){ swap(a[0],a[i]); // max al final
9     bajar(a,i,0); } }
```

(En Java es idéntico al C++ con `a.length`.)

12.10 QuickSelect

Idea: encuentra el k -ésimo menor *sin ordenar todo*. Como quicksort, particiona alrededor de un pivote dejando a la izquierda lo menor y a la derecha lo mayor; pero como sabes en qué lado cae la posición k , recorres (o iteras) sobre *un solo* lado y descartas el otro. Por eso el promedio baja de $O(n \log n)$ a $O(n)$: cada fase procesa la *mitad* del anterior, y $n + \frac{n}{2} + \frac{n}{4} + \dots = 2n$. El peor caso $O(n^2)$ aparece con pivotes pésimos (arreglo casi ordenado + pivote extremo); **elegir el pivote al azar** lo vuelve prácticamente imposible. La versión de abajo es iterativa (sin pila de recursión). Si necesitas garantía $O(n)$ en el peor caso, existe *mediana de medianas* para elegir un buen pivote. **Complejidad:** promedio $O(n)$, peor $O(n^2)$.

```
1 int quickSelect(vector<int> a, int k){ // k in [0,n-1]
2   int lo=0, hi=a.size()-1;
3   while(lo<hi){
4     int p=a[lo+rand()%(hi-lo+1)], i=lo, j=hi; // pivote aleatorio
5     while(i<=j){ while(a[i]<p) i++; while(a[j]>p) j--;
6       if(i<=j){ swap(a[i],a[j]); i++; j--; } }
7     if(k<=j) hi=j; // k a la izq
8     else if(k>=i) lo=i; // k a la der
9     else return a[k];
10  }
11  return a[lo];
12 }
13 // C++ trae nth_element(a.begin(), a.begin()+k, a.end()); // O(n)
```

(En Java igual, con `Random`; código completo en el repo.)

13 Búsqueda Binaria (Binary Search)

Idea central: mantienes un rango $[lo, hi]$ donde *sabes* que está la respuesta y en cada paso lo partes por la mitad, descartando la mitad que no puede contenerla. Como reduces a la mitad cada vez, llegas en $\log_2 n$ pasos. Funciona si los datos están **ordenados** o, más general, si existe un **predicado monótono** (una condición que pasa de falsa a verdadera una sola vez): entonces buscas

la frontera del cambio. **Complejidad:** $O(\log n)$ en discreto, $O(\text{iter})$ en reales, $O(L \log n)$ con cadenas. Tip: usa $\text{lo} + (\text{hi} - \text{lo}) / 2$ para el mid y evitar overflow.

13.1 En arreglo ordenado

Cómo: comparas x con el elemento del medio; si es menor, descartas la mitad derecha, si es mayor la izquierda. `lower_bound` te da el primer índice con valor $\geq x$ y `upper_bound` el primer $> x$; con eso sabes si x existe y cuántas copias hay.

```
1 int i = lower_bound(a.begin(), a.end(), x) - a.begin(); // >= x
2 int j = upper_bound(a.begin(), a.end(), x) - a.begin(); // > x
3 bool hay = binary_search(a.begin(), a.end(), x);
```

13.2 Menor x que cumple

Cómo: aquí no buscas en un arreglo sino en el *rango de respuestas posibles*. El predicado es monótono F.F T.T (una vez que algo “sirve”, todo lo mayor también). En cada paso pruebas el mid: si cumple, lo guardas como candidato y sigues buscando a la izquierda ($\text{hi} = \text{mid} - 1$) por si hay uno menor que también sirva; si no cumple, subes ($\text{lo} = \text{mid} + 1$). Al final `res` es la frontera izquierda: el *menor* que cumple. Ejemplo: “mínima capacidad para repartir en $\leq k$ días”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; hi=mid-1;} // sirve -> izq
4   else lo=mid+1; }
```

13.3 Mayor x que cumple

Cómo: el caso espejo. Ahora el predicado es T..T F..F (sirve hasta cierto punto y luego deja de servir). Cuando el mid cumple, lo guardas y buscas a la *derecha* ($\text{lo} = \text{mid} + 1$) por si hay uno mayor que también sirva; si no cumple, bajas ($\text{hi} = \text{mid} - 1$). Respecto al anterior solo cambian esas dos líneas. Ejemplo: “máxima longitud de corte tal que salgan $\geq k$ piezas”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; lo=mid+1;} // sirve -> der
4   else hi=mid-1; }
```

13.4 Sobre reales

Cómo: cuando la respuesta es un número *real* (raíces, puntos de equilibrio) no hay “mitad entera” ni condición $\text{lo} \leq \text{hi}$ para parar. En su lugar iteras un número fijo de veces (unas 100 basta: el rango se divide por 2^{100} , precisión más que suficiente), moviendo `lo` o `hi` al mid según el predicado. Al terminar, $\text{lo} \approx \text{hi}$ es la frontera buscada.

```
1 for(int it=0; it<100; it++){ double mid=(lo+hi)/2;
2   if(cumple(mid)) hi=mid; else lo=mid; }
3 // lo ~ hi = frontera
```

13.5 Con strings

Cómo: exactamente la misma binaria, pero comparando cadenas en orden *lexicográfico* (diccionario) en lugar de números. La única diferencia de costo es que comparar

dos cadenas cuesta $O(L)$ (longitud), no $O(1)$, así que el total sube a $O(L \log n)$. Ojo: el arreglo debe estar ordenado con el mismo criterio (ASCII: mayúsculas antes que minúsculas).

```
1 // vector<string> s ordenado
2 int i = lower_bound(s.begin(), s.end(), string("caro"))
3   - s.begin();
4 bool hay = binary_search(s.begin(), s.end(), string("caro"));
```

Parte V Matemáticas

14 Matemáticas (Math)

Contenido en desarrollo.

15 Teoría de Números (Number Theory)

Contenido en desarrollo.

16 Combinatoria (Combinatorics)

Contenido en desarrollo.

17 Máscaras de Bits (Bitmasking)

Contenido en desarrollo.

18 Geometría (Geometry)

Contenido en desarrollo.

Parte VI Técnicas Algorítmicas

19 Programación Dinámica (Dynamic Programming)

Contenido en desarrollo.

20 Algoritmos Voraces (Greedy)

Contenido en desarrollo.

21 Ad-hoc y Simulación

Contenido en desarrollo.

Parte VII Cadenas

22 Cadenas (Strings)

Contenido en desarrollo.